With a bit of hard work on CORS, you've managed to get cookies working from the new site. Something tells you that the extra work you needed to do just to get cookies to work is a bad sign. You'd like to mark your cookies as SameSite as a defense in depth against CSRF attacks, but SameSite cookies are incompatible with CORS. Apple's Safari browser is also aggressively blocking cookies on some cross-site requests for privacy reasons, and some users are doing this manually through browser settings and extensions. So, while cookies are still a viable and simple solution for web clients on the same domain as your API, the future looks bleak for cookies with cross-origin clients. You can future-proof your API by moving to an alternative token storage format.

Cookies are such a compelling option for web-based clients because they provide the three components needed to implement token-based authentication in a neat pre-packaged bundle (figure 5.3):

- A standard way to communicate tokens between the client and the server, in the form of the Cookie and Set-Cookie headers. Browsers will handle these headers for your clients automatically, and make sure they are only sent to the correct site.
- A convenient storage location for tokens on the client, that persists across page loads (and reloads) and redirections. Cookies can also survive a browser restart and can even be automatically shared between devices, such as with Apple's Handoff functionality.[2]
- Simple and robust server-side storage of token state, as most web frameworks support cookie storage out of the box just like Spark.